



projeto 3

Programação de Computadores - Experiência Prática 3



2 Identificação do aluno

projeto 2
Programação de Computadores - Experiência
Prática 3

SS

**Samuel dos Santos Escoval CRUZ_EAD_Cst em Análise e Desenvolvimento de
Sistemas_1A_20261**



Rascunho da lógica de palavras pré-definidas para o jogo e o sorteio

Para criar o sistema de palavras do jogo da forca, utilizei a biblioteca random, responsável por permitir a lógica de seleção aleatória dentro do Python. Em seguida, criei uma lista chamada palavras, contendo as possíveis palavras que poderão ser sorteadas durante o jogo, como "PYTHON", "PROGRAMACAO" e "ESTRUTURA". Após isso, defini as regras iniciais do jogo através de variáveis. A variável palavra_secreta utiliza o comando random.choice(palavras) para selecionar aleatoriamente uma palavra da lista criada anteriormente. O trecho de código utilizado foi: `import random` `palavras = [ "PYTHON", "PROGRAMACAO", "ESTRUTURA", "DEPURACAO", "SNAKECASE" ]` `palavra_secreta = random.choice(palavras)`



Definição de estados do jogo

Foi criada a variável `letras_descobertas`, responsável por gerar uma lista contendo "_" na mesma quantidade de letras da palavra sorteada, permitindo esconder inicialmente a palavra do jogador. Além disso, utilizei a variável `palavras_tentadas` com a estrutura `set()` para armazenar as letras já utilizadas pelo usuário, evitando repetições durante a partida. Por fim, foi criada a variável `tentativas`, responsável por definir a quantidade máxima de erros permitidos no jogo. O trecho de código utilizado foi: `#Aqui respondido na pergunta anterior import random palavras = [ "PYTHON", "PROGRAMACAO", "ESTRUTURA", "DEPURACAO", "SNAKECASE" ] palavra_secreta = random.choice(palavras) #Essa parte que se refere a pergunta letras_descobertas = ["_"] * len(palavra_secreta) palavras_tentadas = set() tentativas = 6`



Planejamento das funções do jogo

Você pode responder assim: Para atender aos requisitos de modularização, organizei o jogo em funções com responsabilidades específicas. A função `mostrar_forca()` recebe como entrada o número de `tentativas` e a lista `letras_descobertas`. Ela processa a quantidade de erros cometidos e exibe visualmente a forca, o boneco e o estado atual da palavra. A função `iniciar_jogo()` recebe a lista `palavras` como entrada. Dentro dela, ocorre o sorteio da `palavra_secreta`, a criação da lista `letras_descobertas`, a criação do conjunto `palavras_tentadas` e a definição da quantidade inicial de `tentativas`. Essa função retorna todos esses valores para serem usados no fluxo principal do programa. Também pretendo utilizar a função `processar_tentativa()`, responsável por receber a `letra_usuario`, a `palavra_secreta` e a lista `letras_descobertas`. Ela verifica se a letra digitada existe na palavra e atualiza a posição correta quando houver acerto. Essa divisão facilita o desenvolvimento porque separa cada parte importante do jogo em blocos menores, deixando o código mais organizado, legível e fácil de manter. Além disso, o fluxo principal fica mais simples, pois apenas chama as funções necessárias em cada etapa da partida.



Laços de repetição

Para manter o jogo em execução, utilizo uma variável tentativas com valor inicial igual a 6, representando o número máximo de erros permitidos. A cada letra incorreta digitada pelo jogador, esse valor é reduzido em 1. O controle principal do jogo é feito com um laço while, que continua executando enquanto ainda existirem tentativas disponíveis e enquanto a palavra ainda não tiver sido completamente descoberta. Para verificar se a palavra foi concluída, o programa observa se ainda existe o caractere "_" dentro da lista letras_descobertas. Dessa forma, a condição do laço fica baseada em dois critérios: tentativas > 0 e "_" in letras_descobertas. Enquanto essas duas condições forem verdadeiras, o jogo continua ativo. A cada rodada, o sistema exibe a forca, mostra o estado atual da palavra, informa as tentativas restantes e solicita uma nova letra ao usuário. Também utilizo o conjunto palavras_tentadas para armazenar as letras já digitadas. Assim, o programa consegue verificar se o jogador está tentando repetir uma letra e evitar que essa jogada seja processada novamente. O trecho correspondente do código é: while tentativas > 0 and "_" in letras_descobertas: mostrar_forca(tentativas, letras_descobertas) print(f"Tentativas restantes: {tentativas}") print("Letras tentadas:", palavras_tentadas) letra_usuario = input("Digite uma letra: ").upper() if letra_usuario in palavras_tentadas: print("Você já tentou esta letra") continue palavras_tentadas.add(letra_usuario) A redução das tentativas ocorre quando a letra não pertence à palavra secreta: if letra_usuario not in palavra_secreta: tentativas -= 1 print("Letra incorreta!") Assim, o jogo será encerrado quando o jogador descobrir todas as letras ou quando o número de tentativas chegar a zero.



Planejamento da interatividade

- início do jogo - O sistema apresenta o título do jogo, as regras básicas, informa o número de tentativas e pede que o usuário pressione ENTER para iniciar. Visualização do usuário ===== JOGO DA FORCA ===== Adivinhe a palavra secreta. Você tem 6 tentativas. Digite uma letra por vez. ===== Pressione ENTER para iniciar...
- andamento da partida - Durante cada rodada, o sistema exibe a força, o estado atual da palavra e o número de tentativas restantes. Visualização do Usuário: _____ ||||| _
Palavra: _ _ _ _ _ Tentativas restantes: 6 Digite uma letra:
- Letra repetida - Se o jogador digitar uma letra já utilizada anteriormente, o sistema informa a repetição e solicita uma nova tentativa. Mensagem de exibição : ' Você já tentou a letra A, tente outra.'
- Letra incorreta - Quando o jogador digita uma letra incorreta, o sistema reduz uma tentativa, atualiza o desenho da força e informa quantas tentativas ainda restam. Visualização do Usuário : ----- Letra incorreta! Você tem 5 tentativas restantes. ----- _____ ||| o ||| _
Palavra: _ _ _ _ _ A cada erro, novas partes do boneco são adicionadas à força até completar o desenho final.
- Letra correta - Quando o jogador acerta uma letra, o sistema informa o acerto e atualiza a palavra parcialmente descoberta. Exemplo: ----- Boa! A letra 'A' existe na palavra. ----- _____ ||| o ||| _
Palavra: _ A _ _ A _ Tentativas restantes: 5 Não adiciona partes ao boneco e nem diminui as tentativas
- Vitória - Quando todas as letras forem descobertas, o sistema encerra a partida exibindo a mensagem de vitória, a quantidade de tentativas restantes e a palavra secreta completa. ex: _____ ||| o ||| _
Palavra: E S T R U T U R A Parabéns! Você venceu! Com 5 tentativas restantes. A palavra era: ESTRUTURA
- Derrota - Quando o jogador perde todas as tentativas antes de descobrir a palavra, o sistema encerra a partida exibindo uma mensagem de derrota e revelando a palavra secreta. _____ ||| o | / \ | / \ | _
Palavra: _ _ _ _ _ Game Over! Você perdeu! Tente novamente! A palavra era: SNAKECASE
- Reinício de Jogo - Ao final da partida, o sistema pergunta se o jogador deseja iniciar uma nova rodada. mensagem de input ,se sim retorna ao loop ,e incia um novo jogo: Deseja jogar novamente? (s/n):



Submissão do Jogo da Forca

```

import random # Importa a biblioteca random para selecionar palavras aleatoriamente
def mostrar_forca(tentativas, letras_descobertas): """ Exibe visualmente a forca e o
estado atual da palavra. """ erros = 6 - tentativas # Calcula quantos erros o jogador possui # Estrutura condicional para montar o boneco da forca
cabeca = "o"
if erros >= 1 else " "
tronco = "|"
if erros >= 2 else " "
braco_esquerdo = "/"
if erros >= 3 else " "
braco_direito = "\"
if erros >= 4 else " "
perna_esquerda = "/"
if erros >= 5 else " "
perna_direita = "\"
if erros >= 6 else " "
# Estrutura visual da forca
print("\n _____")
print(" | | ")
print(f" | {cabeca} ")
print(f" | {braco_esquerdo}{tronco}{braco_direito} ")
print(f" | {perna_esquerda} {perna_direita} ")
print(" | | ")
print("_ _ _")
# Mostra o estado atual da palavra
print("Palavra:", " ".join(letras_descobertas))
def iniciar_jogo(palavras): """ Inicializa os dados principais do jogo. """
# Seleciona aleatoriamente uma palavra da lista
palavra_secreta = random.choice(palavras)
# Cria uma lista de "_" com o tamanho da palavra secreta
letras_descobertas = ["_"] * len(palavra_secreta)
# Foi escolhido set porque não permite letras repetidas
palavras_tentadas = set()
# Número máximo de tentativas
tentativas = 6
# Retorna os elementos principais do jogo
return palavra_secreta, letras_descobertas, palavras_tentadas, tentativas
def start_forca(): """ Exibe a tela inicial e as regras básicas do jogo. """
# Interface inicial do jogo
print("\n=====")
print(" JOGO DA FORCA ")
print("=====")
print("Adivinhe a palavra secreta.")
print("Você tem 6 tentativas.")
print("Digite uma letra por vez.")
print("=====\\n")
# Aguarda o usuário iniciar o jogo
input("Pressione ENTER para iniciar...")
def processar_tentativa(letra_usuario, palavra_secreta, letras_descobertas): """ Verifica se a letra digitada existe na palavra secreta. """
# Variável para verificar se houve acerto
acertou = False
# Percorre cada letra da palavra secreta
for posicao, letra in enumerate(palavra_secreta):
# Verifica se a letra digitada corresponde à letra da palavra
if letra == letra_usuario:
# Atualiza a letra correta na posição correspondente
letras_descobertas[posicao] = letra_usuario
# Define que o usuário acertou
acertou = True
# Retorna se o usuário acertou ou não
return acertou
# Lista contendo as palavras possíveis do jogo
palavras = [ "PYTHON", "PROGRAMACAO", "ESTRUTURA", "DEPURACAO", "SNAKECASE" ]
# Variável responsável por controlar novas partidas
jogar_novamente = "s"
# Loop principal do jogo
while jogar_novamente == "s":
# Inicializa os elementos principais da partida
palavra_secreta, letras_descobertas, palavras_tentadas, tentativas = iniciar_jogo(palavras)
# Mostra a tela inicial
start_forca()
# Loop da partida enquanto houver tentativas
while tentativas > 0 and "_" in letras_descobertas:
# Mostra a forca e a palavra atual
mostrar_forca(tentativas, letras_descobertas)
# Mostra quantas tentativas restam
print(f"Tentativas restantes: {tentativas}")
# Solicita uma letra ao usuário
letra_usuario = input("Digite uma letra: ").upper()
# Verifica se a letra já foi utilizada
if letra_usuario in palavras_tentadas:
print(f"Você já tentou a letra {letra_usuario}, tente outra.")
continue
# Adiciona a letra ao conjunto de letras tentadas
palavras_tentadas.add(letra_usuario)
# Processa a tentativa do usuário
acertou = processar_tentativa( letra_usuario, palavra_secreta, letras_descobertas )
# Executa caso o usuário erre
if not acertou:
# Reduz uma tentativa
tentativas -= 1
# Mensagem de erro
print("\n-----")
print(f"Letra incorreta! Você tem {tentativas} tentativas restantes.")
print("-----")
# Executa caso o usuário acerte
else:
# Mensagem de acerto
print("\n-----")
print(f"Boa! A letra '{letra_usuario}' existe na palavra.")
print("-----")
# Condição de vitória
if "_" not in letras_descobertas:
# Mostra o estado final da forca
mostrar_forca(tentativas, letras_descobertas)
# Mensagem de vitória
print(f"\nParabéns! Você venceu! Com {tentativas} tentativas restantes.")
# Mostra a palavra secreta
print("A palavra era:", palavra_secreta)
# Condição de derrota
else:
# Mostra a forca final
mostrar_forca(tentativas, letras_descobertas)
# Mensagem de derrota
print("\nGame Over! Você perdeu! Tente novamente!")
# Mostra a palavra secreta
print("A palavra era:", palavra_secreta)
# Pergunta se o usuário deseja jogar novamente
jogar_novamente = input("\nDeseja jogar novamente? (s/n): ").lower()

```




Verificação das atividades realizadas

- Importei e utilizei a biblioteca '*random*' para sorteio da palavra.
- Criei uma lista de palavras para o banco de opções do jogo.
- Implementei um conjunto (*set*) para evitar letras duplicadas nas tentativas do usuário.
- Utilizei uma lista para representar adequadamente a palavra adivinhada e suas lacunas ('_').
- O jogo possui controle coerente do número de tentativas restantes.
- O laço principal ('*while*') gerencia corretamente o andamento do jogo e a interrupção da partida.
- O código está modularizado em funções com responsabilidades diferentes.
- A interface de texto oferece *feedback* amigável a cada jogada.
- O código submetido contém comentários explicativos e *docstrings* formatadas em suas funções.
- Variáveis e funções foram nomeadas utilizando a padronização '*snake_case*'.



Reflexão

Durante o desenvolvimento desta experiência prática, consegui atender os requisitos propostos pelo exercício. No início, tive dificuldades relacionadas à construção da lógica do programa, pois ainda estou desenvolvendo meu raciocínio lógico em Python. Para facilitar o entendimento, dividi o projeto em pequenas etapas, começando pela importação da biblioteca random, criação da lista de palavras e desenvolvimento do laço principal com while. Após testar o funcionamento inicial do jogo, percebi que a lógica estava correta e decidi melhorar a interface textual. Implementei uma representação visual da força e do boneco utilizando estruturas condicionais, além de organizar melhor o código através da criação de funções com def, separando as responsabilidades do sistema. Também melhorei as mensagens exibidas ao usuário, tornando a interação mais clara e dinâmica. Durante a implementação da opção de reiniciar o jogo, encontrei dificuldades relacionadas ao controle do while e à indentação do Python. Em alguns momentos, criei loops incorretos que repetiam telas ou impediam a exibição de mensagens. Com testes e correções sucessivas, compreendi melhor a importância da organização lógica e da indentação correta no funcionamento do programa. Como ponto forte, destaco minha capacidade de dividir o problema em partes menores e testar constantemente o sistema. Como oportunidade de melhoria, reconheço que ainda preciso desenvolver mais segurança na criação de loops e no planejamento inicial da lógica do programa. De forma geral, esta experiência prática contribuiu muito para meu desenvolvimento acadêmico e profissional, fortalecendo minha compreensão sobre lógica computacional, modularização e organização de código em Python.

